

Introducción a Numpy

Clase 12

Numpy

NumPy, que significa "Numerical Python", es una biblioteca fundamental en el ecosistema de Python para el cálculo numérico y científico. Proporciona un objeto de matriz multidimensional eficiente llamado "ndarray" (arreglo multidimensional de NumPy), que es más eficiente en términos de memoria y tiempo de ejecución que las listas estándar de Python

Ventajas

NumPy es ampliamente utilizado en el procesamiento de datos, la ciencia de datos, el aprendizaje automático, la visualización de datos y otras áreas que involucran cálculos numéricos. Algunas de las ventajas clave de NumPy incluyen:

- **Eficiencia:** Los arreglos NumPy están optimizados para realizar operaciones numéricas rápidas y eficientes. Internamente, los arreglos NumPy están almacenados de manera contigua en la memoria, lo que permite realizar operaciones vectorizadas en lugar de bucles explícitos.
- **Funcionalidades matemáticas:** NumPy proporciona una amplia gama de funciones matemáticas y operaciones para realizar cálculos avanzados en arreglos. Estas funciones incluyen trigonometría, álgebra lineal, estadísticas, transformadas, funciones especiales y más.
- **Broadcasting:** NumPy permite realizar operaciones entre arreglos de diferentes formas y dimensiones mediante el broadcasting. Esto simplifica el proceso de realizar cálculos entre arreglos que no tienen las mismas dimensiones y permite escribir código más conciso.
- **Integración con otras bibliotecas:** NumPy es la base para muchas otras bibliotecas populares de Python utilizadas en ciencia de datos y aprendizaje automático, como Pandas, Matplotlib y Scikit-learn. Estas bibliotecas aprovechan las capacidades de NumPy para el manejo eficiente de datos y cálculos numéricos.

Arreglos numpy

Los arreglos NumPy, o ndarray (n-dimensional array), pueden contener elementos del mismo tipo y tienen una forma (shape) que define la estructura del arreglo.

```
import numpy as np
```

```
# Creación de un arreglo unidimensional
```

```
arr1d = np.array([1, 2, 3, 4, 5])
```

```
print(arr1d) # Salida: [1 2 3 4 5]
```

```
# Creación de un arreglo bidimensional
```

```
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(arr2d)
```

```
# Salida:
```

```
# [[1 2 3]
```

```
# [4 5 6]]
```

Propiedades importantes

- **Shape:** Representa las dimensiones del arreglo y devuelve una tupla de enteros que indica el tamaño de cada dimensión. Por ejemplo, un arreglo de forma (2, 3) tiene 2 **filas** y 3 **columnas**.
- **Size:** Indica el número total de elementos en el arreglo.
- **Dtype:** Especifica el tipo de datos de los elementos del arreglo. NumPy admite varios tipos de datos, como enteros, flotantes, booleanos, etc.

```
import numpy as np
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])  
print(arr.shape) # Salida: (2, 3)  
print(arr.size) # Salida: 6  
print(arr.dtype) # Salida: int64  
print(arr.ndim) # Salida: 2
```

Arrays Homogéneos y Heterogéneos

Arrays Homogéneos

```
import numpy as np
```

```
homogeneous_array = np.array([1, 2, 3, 4, 5])
```

Ventajas:

Eficiencia en operaciones numéricas.
Optimizaciones de tipo para mejor rendimiento.

Arrays Heterogéneos

```
import numpy as np
```

```
het_array = np.array([1, 'dos', 3.0, [4, 5]], dtype=object)
```

Consideraciones:

Pérdida de eficiencia debido a la falta de optimizaciones de tipo.
Mayor flexibilidad en la composición de datos.

Acceso y manipulación de elementos individuales

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])  
print(arr[0]) # Salida: 1
```

```
arr[2] = 6  
print(arr) # Salida: [1 2 6 4 5]
```

```
import numpy as np
```

```
arr = np.array([[1, 2, 3], (4, 5, 6)])  
print(arr[0, 1]) # Salida: 2
```

```
arr[1, 2] = 7  
print(arr)  
# Salida:  
# [[1 2 3]  
#  [4 5 7]]
```

Slicing para extraer subarreglos

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])  
print(arr[1:4]) # Salida: [2 3 4]
```

```
arr_slice = arr[:,2]  
print(arr_slice) # Salida: [1 3 5]
```

```
import numpy as np
```

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
fila0 = arr[0, :]  
print(fila0)  
# Salida: [1 2 3]
```

```
colu0 = arr[:, 0]  
print(colu0)  
# Salida: [1 4 7]
```

```
arr_slice = arr[0:2, 1:]  
print(arr_slice)  
# Salida:  
# [[2 3]  
# [5 6]]
```


Indexación booleana

La indexación booleana permite filtrar elementos en un arreglo NumPy basado en una condición booleana. Se utiliza una matriz de valores booleanos o una expresión booleana para indexar el arreglo.

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
mask = arr > 3
```

```
print(arr[mask]) # Salida: [4 5]
```

```
mask = (arr % 2 == 0)
```

```
arr[mask] = 0
```

```
print(arr) # Salida: [1 0 3 0 5]
```

Operaciones aritméticas básicas

```
import numpy as np
```

```
arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([4, 5, 6])
```

```
suma = arr1 + arr2
```

```
resta = arr1 - arr2
```

```
multiplicacion = arr1 * arr2
```

```
division = arr1 / arr2
```

```
print(suma)      # Salida: [5 7 9]
```

```
print(resta)     # Salida: [-3 -3 -3]
```

```
print(multiplicacion) # Salida: [4 10 18]
```

```
print(division)  # Salida: [0.25 0.4 0.5]
```

```
print(arr1 + 1)  # Salida: [2 3 4]
```

```
print(arr1 * 2)  # Salida: [2 4 6]
```

Broadcasting

Broadcasting es una poderosa funcionalidad de NumPy que permite realizar **operaciones entre arreglos de diferentes formas y dimensiones**, sin necesidad de que tengan las mismas dimensiones. NumPy realiza automáticamente la expansión de dimensiones necesaria para que las operaciones sean compatibles.

```
a = np.array([[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]])
```

```
b = np.array([1, 0, 1])
```

```
c = a + b # Suma los valores de b a todas las líneas de a
```

```
print(c)
```

```
# Salida:
```

```
# [[ 2  2  4]
```

```
# [ 5  5  7]
```

```
# [ 8  8 10]
```

```
# [11 11 13]]
```

Transposición de arreglos

La transposición de un arreglo NumPy implica intercambiar filas por columnas. Esto se logra utilizando el atributo `T` del arreglo o la función `numpy.transpose()`. La transposición es útil para cambiar la forma de los arreglos o realizar operaciones matriciales.

```
import numpy as np
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
transpuesta = arr.T
```

```
print(arr)      # Salida: [[1 2 3]
```

```
                #      [4 5 6]]
```

```
print(transpuesta) # Salida: [[1 4]
```

```
                #      [2 5]
```

```
                #      [3 6]]
```

Concatenación

La concatenación de arreglos en NumPy se realiza utilizando las funciones `numpy.concatenate()`, `numpy.vstack()` y `numpy.hstack()`. Estas funciones permiten combinar arreglos a lo largo de diferentes dimensiones.

```
arr1 = np.array([[1, 2, 3], [4, 5, 6]])  
arr2 = np.array([[7, 8, 9], [10, 11, 12]])
```

```
concatenacion = np.concatenate((arr1, arr2))  
vertical = np.vstack((arr1, arr2))  
horizontal = np.hstack((arr1, arr2))
```

```
print(concatenacion) # Salida: [[ 1  2  3]  
print(vertical)      #      [ 4  5  6]  
                    #      [ 7  8  9]  
                    #      [10 11 12]]  
print(horizontal)    # Salida: [[ 1  2  3  7  8  9]  
                    #      [ 4  5  6 10 11 12]]
```

```
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])
```

```
concatenacion = np.concatenate((arr1, arr2))  
vertical = np.vstack((arr1, arr2))  
horizontal = np.hstack((arr1, arr2))
```

```
print(concatenacion) # Salida: [1 2 3 4 5 6]  
print(vertical)      # Salida:  
                    # [[1 2 3]  
                    #  [4 5 6]]  
print(horizontal)    # Salida: [1 2 3 4 5 6]
```

Manipulación de dimensiones y remodelación de arreglos

NumPy ofrece varias funciones para manipular las dimensiones y remodelar arreglos. Estas funciones incluyen `numpy.reshape()`, `numpy.resize()`, `numpy.expand_dims()`, `numpy.squeeze()` y más. Estas funciones permiten cambiar la forma de los arreglos de acuerdo a las necesidades del problema.

```
arr = np.array([1, 2, 3, 4, 5, 6])
```

```
remodelado = np.reshape(arr, (2, 3))
```

```
ampliado = np.expand_dims(arr, axis=1)
```

```
reducido = np.squeeze(ampliado)
```

```
print(remodelado)
```

```
# Salida:
```

```
# [[1 2 3]
```

```
# [4 5 6]]
```

```
print(ampliado)
```

```
# Salida:
```

```
# [[1
```

```
# [2]
```

```
# [3]
```

```
# [4]
```

```
# [5]
```

```
# [6]]
```

```
print(reducido) # Salida: [1 2 3 4 5 6]
```

Algunas funciones

```
import numpy as np
```

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
suma_total = np.sum(arr)
```

```
suma_por_col = np.sum(arr, axis=0)
```

```
minimo_por_fil = np.min(arr, axis=1)
```

```
maximo_total = np.max(arr)
```

```
media = np.mean(arr)
```

```
print(suma_total)    # Salida: 21
```

```
print(suma_por_col)  # Salida: [5 7 9]
```

```
print(minimo_por_fil) # Salida: [1 4]
```

```
print(maximo_total)  # Salida: 6
```

```
print(media)         # Salida: 3.5
```

multiplicación matricial y producto escalar.

```
import numpy as np
```

```
# Matrices de ejemplo
```

```
matriz_a = np.array([[1, 2], [3, 4]])
```

```
matriz_b = np.array([[5, 6], [7, 8]])
```

```
# Multiplicación matricial
```

```
resultado = np.dot(matriz_a, matriz_b)
```

```
print("Matriz A:\n", matriz_a)
```

```
print("Matriz B:\n", matriz_b)
```

```
print("Resultado de np.dot:\n", resultado)
```

```
import numpy as np
```

```
# Arrays de ejemplo
```

```
array_a = np.array([1, 2, 3])
```

```
array_b = np.array([4, 5, 6])
```

```
# Producto escalar
```

```
resultado_escalar = np.dot(array_a,  
array_b)
```

```
print("Array A:", array_a)
```

```
print("Array B:", array_b)
```

```
print("Resultado de np.dot:",  
resultado_escalar)
```


Generar arrays

```
# ceros
a = np.zeros((2,3))
# [[0. 0. 0.]
# [0. 0. 0.]]

# unos
b = np.ones((2,3))
# [[1. 1. 1.]
# [1. 1. 1.]]

# valor específico
c = np.full((3,3),5.0)
# [[5. 5. 5.]
# [5. 5. 5.]
# [5. 5. 5.]]
```

```
# identidad
d = np.eye(3)
# 3x3
# [[1. 0. 0.]
# [0. 1. 0.]
# [0. 0. 1.]]

# rango
e = np.arange(10)
# [0 1 2 3 4 5 6 7 8 9]
```

Random

```
a = np.random.random((3,2)) # números entre 0-1  
# [[0.06121857 0.10180167]  
# [0.83321726 0.54906613]  
# [0.94170273 0.19447411]]
```

```
# el menor no es necesario 0-x  
b = np.random.randint(3,10, size=(3,3))  
# [[6 8 4]  
# [3 6 3]  
# [4 7 8]]
```

```
# elige elementos al hazar  
c = np.random.choice([1,2,3,4], size=8)  
# [4 2 1 3 4 1 4 1]
```

Ejercicio 1

Obtener la primera columna y sumar 10 a cada elemento.

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
# Salida: [11 14]
```

Ejercicio 2

Obtener los elementos pares y concatenarlos con los elementos impares.

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

```
# Salida: [2 4 6 8 10 1 3 5 7 9]
```

Ejercicio 3

Multiplicar cada elemento de arr1 ([[1, 2], [3, 4]]) por una matriz arr2 (array de 1f, 2c con números aleatorios entre el 0 y 1) utilizando broadcasting.

```
arr1 = np.array([[1, 2], [3, 4]])
```

Ejercicio 4

Obtener la submatriz formada por las dos primeras filas y calcular la suma total.

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Salida: 21

Ejercicio 5

Obtener los elementos mayores que la media e imprimirlos.

```
arr = np.array([1, 2, 3, 4, 5, 6])
```

```
# Salida: [4 5 6]
```

Ejercicio 5

Generar 2 arrays con 3 números enteros aleatorios entre el 1 y 10. Concatenarlos y hallar el mínimo.

Ejercicio 6

Dividir cada elemento de la segunda fila entre el elemento de la primera fila correspondiente. Calcular el promedio del resultado y multiplicarlo por el arr original

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```

```
# Salida: [[ 2.83333333  5.66666667  8.5      ]
```

```
         [11.33333333 14.16666667 17.      ]]
```

```
# print(np.round(res, decimals=2))
```

```
# [[ 2.83  5.67  8.5 ]
```

```
   [11.33 14.17 17.  ]]
```

Ejercicio 7

Obtener los elementos mayores que 2, multiplicarlos por 3 y calcular su suma.

```
arr = np.array([1, 2, 3, 4, 5])
```

Salida: 36

Ejercicio 8

Obtener los elementos únicos (`np.unique(arr)`) de un array unidimensional de 20 números generados aleatoriamente entre el 1 y el 20 y ordenarlos de forma descendente.